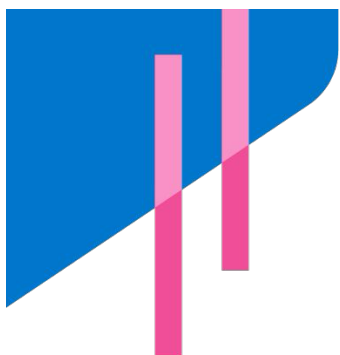
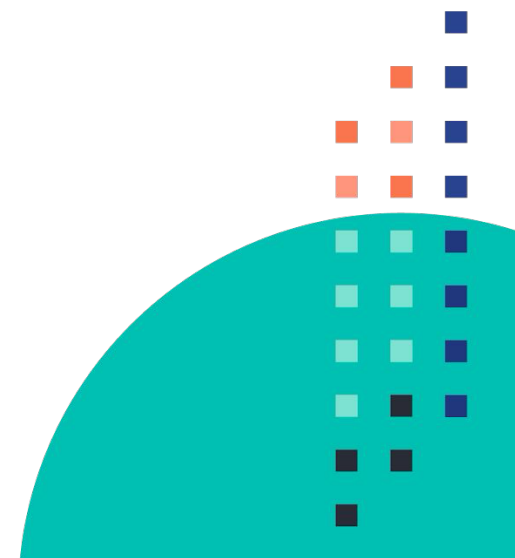




AI open telemetry from **Local Agents (LLMs)** to **ElasticCloud**

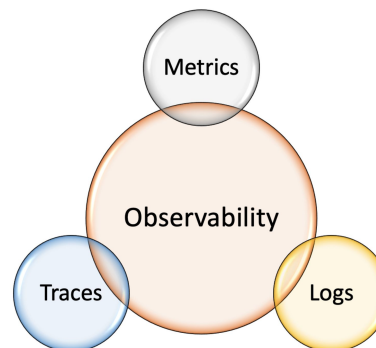
Presenter: Bhuvan





What is Observability

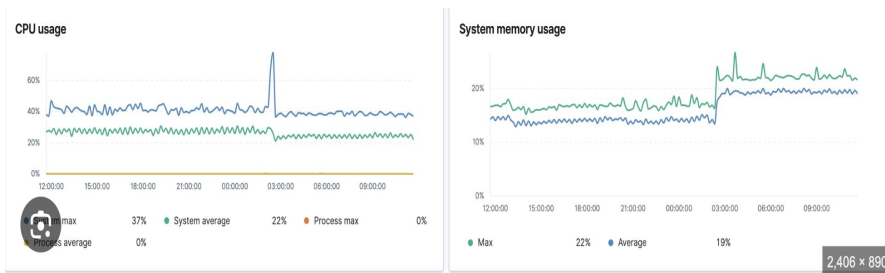
- In the context of modern application development, observability refers to the collection and analysis of data — logs, metrics and traces — from a wide variety of sources, to provide detailed insight into the behavior of applications running in your environments. It can be applied to any system that you build and want to monitor.
- **Observability helps organizations to be proactive rather than like regular active monitoring.**





Logs:

Log is an record of a discrete event that happened in a system at any point of time during the request life cycle. A log usually includes the timestamp and context payload for an event and can be emitted in various forms like plain text, structured format like json or binary logs.



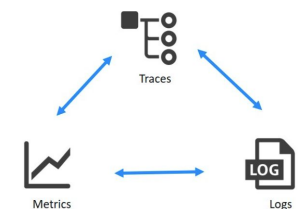
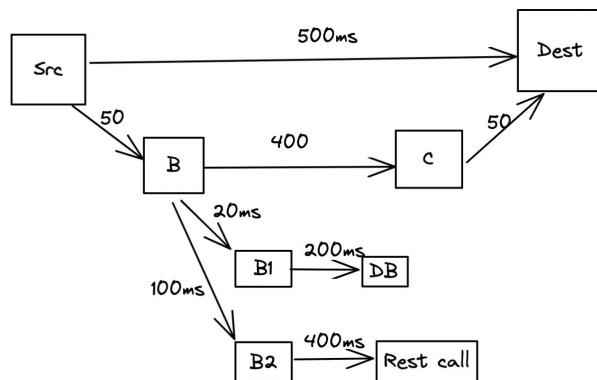
Metrics:

Metrics are the numeric representation like percentiles or the averages which helps in monitoring the fate of a system holistically and are measured over an interval of time. Metric usually is composed of fields like Metric name(CPU, Memory, disk etc), Timestamp and Labels

Traces:

Tracing gives the capability to monitor the fate of a request during its lifecycle across various components in a system.

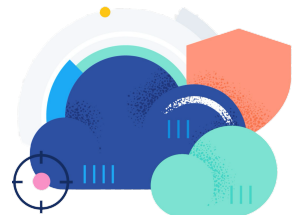
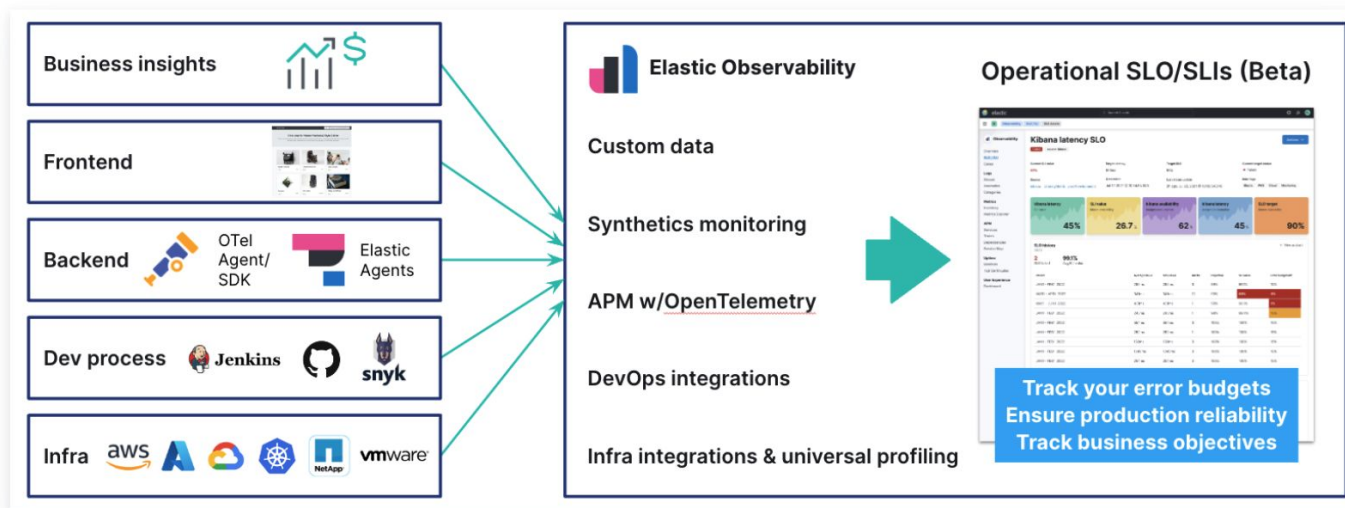
Note:Modern observability also includes additional signal types: **profiling** (continuous CPU/memory flamegraphs), **events** (discrete state changes), and **real user monitoring / RUM** (browser/mobile experience data)





Why we need Elastic Observability

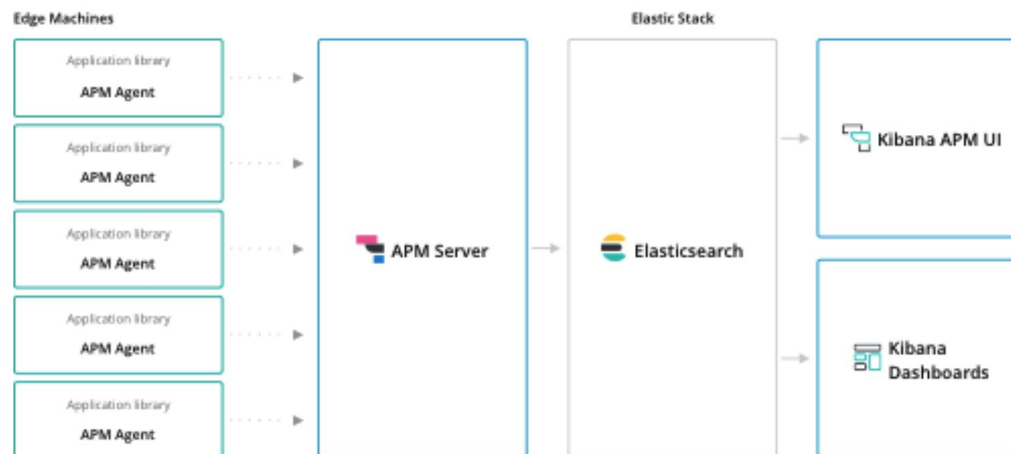
- Unified Search-First Platform – Search logs, metrics, and traces in a single, lightning-fast data store to find root causes in seconds.
- Integrated Security (SIEM) – Use the same data for both IT operations and security threat detection, breaking down team silos.
- Cost-Efficient Data Tiering – Save money by moving older data to "cold" or "frozen" storage while keeping it fully searchable for compliance.
- Zero Vendor Lock-in – Built on OpenTelemetry (OTel) standards, allowing you to ingest data from anywhere with/without proprietary agents.





General APM — agent/sdk-based application monitoring

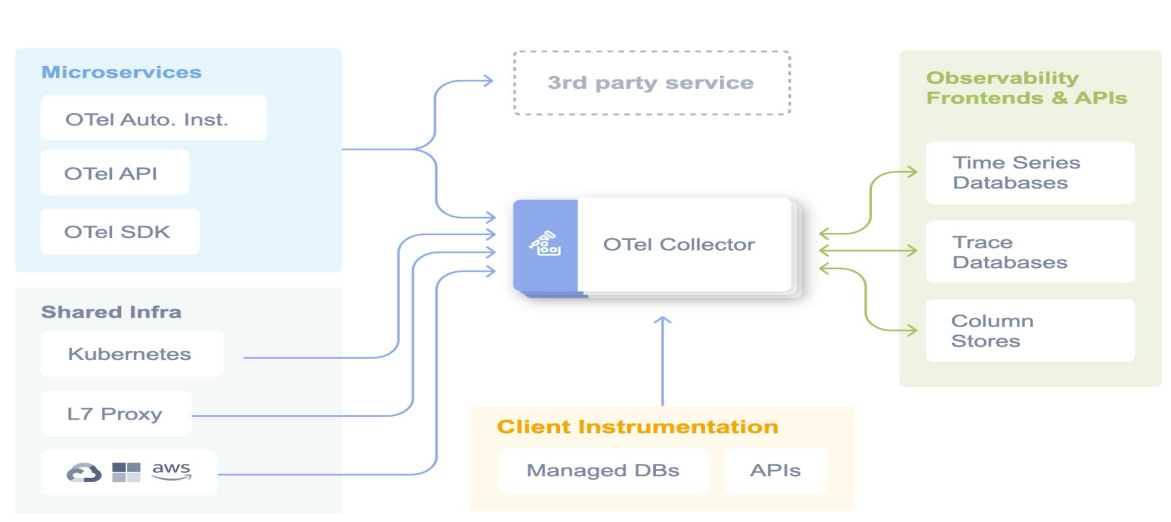
- APM is how teams operate software in production: slow requests, errors, dependencies — usually by analyzing distributed traces and service relationships (waterfall / service map style views).
- In many organizations, “APM” is the first place engineers go when users report latency or failures — even before they say the word “observability.”
- you need an Vendor specific agent or vendor specific SDK to be able to send data to the endpoint for monitoring.





Understanding of OTEL within Observability

- OpenTelemetry (OTel) is an open source observability framework for collecting, processing, and exporting telemetry data (logs, metrics, and traces) in a single, unified format.
- It was developed by the Cloud Native Computing Foundation ([CNCF](#)) to standardize how telemetry data is collected and sent to observability backends.



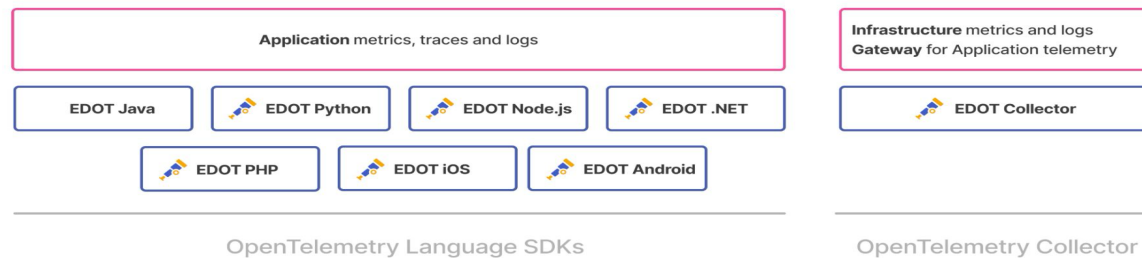


Elastic Observability with OTEL

- [Elastic Observability](#) seamlessly integrates OpenTelemetry data and adds powerful search, analytics, machine learning, and visualization at scale. In 2023, Elastic itself contributed ECS to OpenTelemetry to help unify telemetry data formats.(helps with vendor neutrality)
- One platform for signals: Bring traces, metrics, and logs into Elasticsearch and analyze them in Kibana—reduce tool sprawl for cloud-native and distributed systems (including GenAI services you operate).
- Elastic ingestion path: Send OTLP to Elastic (for example Elastic Cloud Managed OTLP endpoint) so telemetry lands in a scalable datastore built for high-cardinality observability data.
- [Elastic Distributions of OpenTelemetry \(EDOT\)](#) provides SREs and developers with a stable OTel ecosystem. With Elastic's OTel-first approach preserving the OpenTelemetry-native schematic conventions, no schema conversions are needed.



Why EDOT



- EDOT components build on upstream OpenTelemetry; telemetry is still OpenTelemetry + OTLP with semantic conventions preserved for ingestion.
- Upstream OTel requires picking SDK pieces + Collector contrib modules with mixed maturity; EDOT provides opinionated defaults and auto-instrumentation.

For example, in environments like .NET or Java, you can capture traces, metrics, and logs with zero code changes, significantly speeding up your time-to-value

- EDOT additionally includes fixes beyond OTel release cycles and enterprise-grade support without proprietary add-ons.
- EDOT is still OpenTelemetry—you're not adopting a proprietary trace wire format in place of OTel; signals remain exportable as OTLP.
- EDOT remains 100% standards-compliant. Elastic commits all its enhancements back to the [OpenTelemetry project](#), ensuring you have a path to move back to "vanilla" OTel if you ever choose to switch or being neutral.





What is AI observability?

- GenAI observability is the practice of monitoring and troubleshooting health, performance, quality, and safety of applications that call LLMs and agent workflows — not only infrastructure metrics.
- Unlike purely deterministic apps, LLM systems can be non-deterministic: the same input may yield different outputs, so “green dashboards” alone can hide bad outcomes.
- Operational implication: you need technical signals (latency/errors), usage signals (tokens/cost), traceability (prompt/tool/RAG chain), and quality evaluation (beyond HTTP 200).

for example: A request can return HTTP 200 while producing wrong or harmful content — success codes don’t imply answer quality.

- Air-gapped and on-prem model deployments (Ollama, vLLM, fine-tuned internal models) are increasingly common in production

The key insight: only telemetry leaves your environment — your prompts, responses, and model weights stay internal

- Elastic observability works identically whether your model is local, cloud, or hybrid — the instrumentation pattern doesn't change



Using Elastic EDOT for GenAI

- GenAI behaves like distributed software: latency spikes, retries, provider/API errors, tool failures.
- Elastic brings traces, metrics, and logs into one operational view.
- Elastic Cloud exposes a Managed OTLP endpoint so apps send telemetry over standard OTLP without standing up a separate trace backend for the lab.
- same platform for Java / .NET / Python inference services, BFFs, and worker pipelines calling models.

Local-to-Cloud Observability Pipeline

Python Client



Ollama (localhost)



EDOT / OTel



Elastic OTLP

Telemetry sent via Managed OTLP endpoint using API Key Authorization.



Why running within Elastic Cloud?

- Managed OTLP endpoint: send OTLP over HTTPS to Elastic Cloud using standard API key auth — less to run yourself than assembling trace backends and collectors for early phases.
- Operational simplicity: Elastic operates scaling and ingestion plumbing for OTLP so teams focus on instrumentation and SRE workflows (dashboards, alerts, SLOs) in Kibana.
- Single datastore story: traces/logs/metrics land in Elasticsearch-backed workflows — fewer stitched tools for correlation during incidents.
- Not a lock-in of OpenTelemetry: instrumentation stays OTel + OTLP; Elastic Cloud is the managed destination you operate against for this workshop and many production paths.



Now lets see AI Otel in action



Configuration



Environment & OTLP Auth

- Ollama (Terminal A): runs the local model and exposes an OpenAI-style API at <http://localhost:11434/v1>.
- Python (Terminal B): use a virtual environment so packages for this lab don't clash with the rest of your Mac.
- `.env` file: Ollama mode points the OpenAI client to localhost; you don't need a paid cloud key for this path.
- Elastic export: set `OTEL_EXPORTER_OTLP_ENDPOINT` and `OTEL_EXPORTER_OTLP_HEADERS` (from Kibana) so telemetry goes to your Elastic Cloud project.

```
OTEL_EXPORTER_OTLP_ENDPOINT=https://[YOUR_ENDPOINT]
OTEL_EXPORTER_OTLP_HEADERS=Authorization=ApiKey [YOUR_SECRET_KEY]
OPENAI_BASE_URL=http://localhost:11434/v1
```

Here we are using Elastic Cloud Managed OTLP ingestion for secure, high-speed telemetry collection.



Instrumentation



Minimal Code Changes

Add observability to the app

- App code stays familiar: the script uses the OpenAI Python client; in the lab it talks to Ollama via the URL in `.env`.
- Auto-instrumentation: use `elastic-opentelemetry`— either wrap the command with `opentelemetry-instrument` or start OTel first in the app (workshop pattern).
- What you get without rewriting chat logic: traces (and related logs/events) around the model call.
- Why: we observe our process, not someone else's website — same pattern works when you switch to a cloud model later.

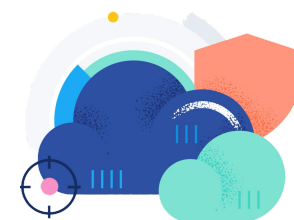
```
# Execute with zero-code instrumentation
export OTEL_PYTHON_LOGGING_AUTO_INSTRUMENTATION=true
opentelemetry-instrument python main.py
```



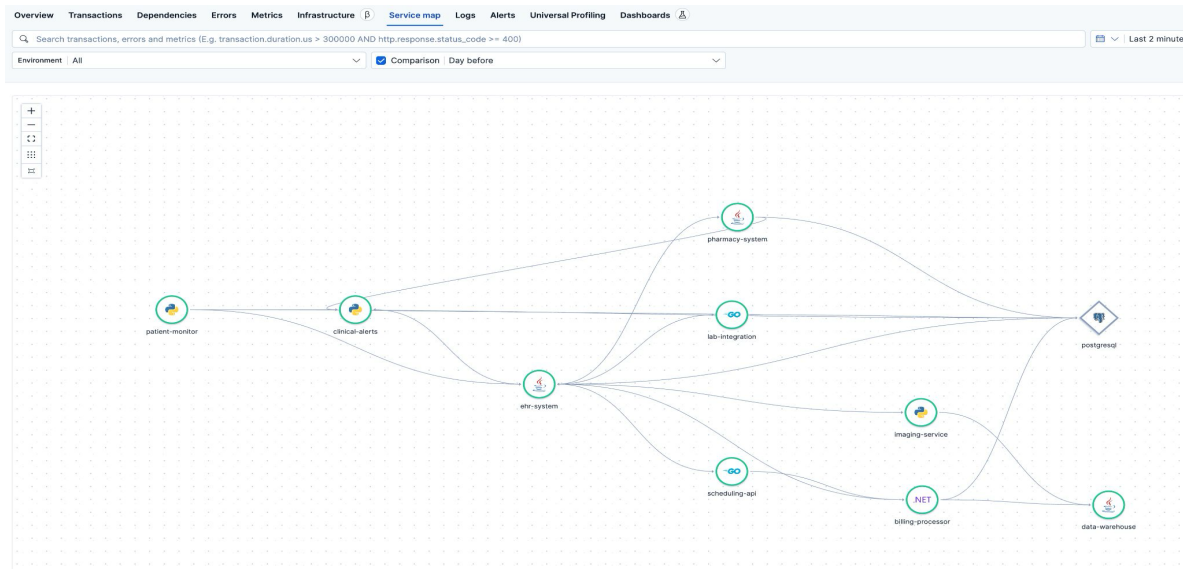
Viewing observability in Kibana



- Open your Elastic Observability project and set time to Last 15 minutes.
- Go to Traces (APM / OTLP view) and find `service.name: testing-genai`.
- Open one trace and walk the group through: total time and what the span(s) represent.
- Optional: in Discover, look for `gen_ai.*` style events if your data includes them — useful for SLO ideas later.



Example Screenshots in Prod for Traces and Servicemap



Conclusion / "Try It Yourself"



- GenAI calls are just distributed service calls — OTEL + Elastic gives you the same operational visibility you already use for APIs and microservices
- Local Ollama today → production cloud model tomorrow: the instrumentation pattern is identical, only the endpoint changes
- Start with traces and key logs; layer in metrics, SLOs, and alerts as your GenAI service matures
- Everything shown today is open standard — no proprietary agents, no vendor lock-in on your instrumentation
- Try it yourself: [your GitHub repo or workshop link here] — spin up Ollama, install EDOT, and see traces in Elastic Cloud in under 30 minutes

Ref:

<https://medium.com/@bhuvan.tirumuneedi/beginner-lab-observe-genai-calls-with-opentelemetry-and-elastic-cloud-ollama-openai-72fbc51a18a4>

<https://github.com/elastic/testing-genai-applications>